

Audio Deep Learning

Assignment #2

Solve the Audio Classification

Assignment overview and motivation

- *Format: Individual or group (max 3 students) with offline defense*
- *Grading: 17 points*
- *Timeline: Assignment # 2 available on 6th June, Code submission deadline till 21th June, and offline defense deadline till 5th July*
- **Submission via Google Form:** <https://forms.gle/o3z2kxHUfiXSZY3C9>

While completing this assignment, you will develop and strengthen the following skills:

- *Approaching a real-world neurocritical care problem by applying digital signal processing techniques to transform raw 1D EEG time-series data into spectrograms.*
- *Conducting focused EDA on physiological signals to analyze frequency bands, temporal dynamics, and signal-to-noise ratios, mapping how different forms of expert agreement correlate with distinct patterns in the data.*
- *Building a reliable Cross-Validation strategy that strictly accounts for the temporal nature of the recordings, patient groupings, and label uncertainty, while monitoring its correlation with the Kaggle Public Leaderboard.*
- *Adapting CNNs and ViTs to effectively process complex, spectrogram-based medical representations for multi-class probability prediction (optimizing for Kullback-Leibler Divergence).*
- *Performing ablation studies not only on model architectures but also on the signal processing pipeline itself (e.g., filtering methods, window sizes, and spectrogram generation techniques) to scientifically isolate the best approach.*
- *Implementing and evaluating speaker diarization pipelines using frame-level speaker activity detection and permutation-invariant training, while assessing performance with standard diarization metrics such as Detection Error Rate (DER).*

Task

Option 1

Task Selection:

- Take part in the [dedicated Kaggle In-Class competition](#)

EDA and Metric Analysis:

- Perform Exploratory Data Analysis (EDA).
- Analyze the proposed target metric: discuss its strengths, weaknesses, and potential edge cases.
- Suggest complementary metrics to capture other aspects of performance.
- Propose an alternative target metric, with justification.

Validation:

- Design a validation strategy based on common sense and expected correlation with Public leaderboard scores.
- Explain your motivation for the chosen strategy.
- Discuss possible reasons for low or high correlation with leaderboard results.

Deep Classification Neural Nets:

- Explore different neural networks to solve the classification problem
- Collect all relevant results:
 - metrics
 - hard samples analysis.
- Submit predictions and compare Public leaderboard metrics with local validation results.
- Conclude whether the model is adequate.
- **Prepare an ablation study on the tested models and methods**

Final Report

- Prepare a comprehensive report including:
 - data insights,
 - feature engineering,
 - tried algorithms and models,
 - chosen hyperparameters,
 - metrics,
 - insights and conclusions.
- Reports may be prepared in LaTeX or Markdown ([README.md](#)).

Option 2

Implement **both** approaches to build a speaker diarization, speaker verification, and voice activity detection system:

Pyannote-Based System

- Use the [VoxCeleb dataset](#).
- Download ready-to-use RTTM annotation files from [GitHub](#) along with their corresponding .wav files.
- Create a UEM file (guided by common sense) and modify the AMI-diarization-setup/pyannote/database.yml configuration file to point to the correct directories.
- Fine-tune the segmentation model from the pyannote pipeline. Use the dev split for training/validation and the test split for testing.

Custom Implementation (PyTorch / Torchaudio / PyTorch-Lightning)

- Use either the AMI (from lecture) or [VoxCeleb dataset](#).
- Build the following from scratch:
 1. A Voice Activity Detection (VAD) model using a supervised neural network approach.
 2. A segmentation model utilizing permutation invariant CE loss.
 3. Audio embeddings for speaker verification / identification.
- **Important: For this step, you are strictly forbidden from using high-level audio frameworks like pyannote or NeMo. You may use torchvision if necessary.**

EDA and Metric Analysis

- Perform Exploratory Data Analysis (EDA): Investigate audio duration distributions, class balance (speech vs. silence), the number of speakers per recording, etc.
- Analyze the target metric: Evaluate your model's performance using the Diarization Error Rate (DER) metric. Discuss its strengths, weaknesses, and potential edge cases (such as overlapping speech from multiple speakers).
- Suggest complementary metrics: Propose metrics to capture other performance aspects (e.g., False Alarm and Missed Speech for VAD, or EER/MinDCF for speaker verification).
- Propose an alternative target metric: Provide a justified alternative metric for the task.

Validation

- Design a validation strategy: Create a robust data-splitting strategy (e.g., fixing a dev set for training/validation and a test set for final evaluation) based on common sense and expected correlation with final test results. **If an already predefined test set is provided - use it as a test set, not validation (design validation by yourself)**
- Explain your motivation: Justify why you chose this specific validation strategy.

Final Report

- Prepare a comprehensive report including:
 - data insights,
 - feature engineering,
 - tried algorithms and models,
 - chosen hyperparameters,
 - metrics,
 - insights and conclusions.
- Reports may be prepared in LaTeX or Markdown ([README.md](#)).

NOTES

- Do not hesitate to include failed ideas or experiments in your report. Documenting unsuccessful attempts is a strong indication that you systematically explored different approaches and gradually arrived at the best-performing solution, rather than relying on a “lucky shot.” Moreover, analyzing failed approaches can often lead to valuable insights about the dataset, the task itself, and the behavior of deep learning algorithms. Such reflections may also highlight potential directions for future improvements and demonstrate a deeper understanding of the problem-solving process.
- Keep your EDA concise. Avoid overloading it with excessive plots, histograms, or tables. Focus only on key aspects of the data (e.g., missing values, target distribution). If a part of your EDA does not lead to meaningful conclusions, you likely don't need it (for example, full pairwise correlation matrices). Do NOT follow the typical Kaggle-style EDA – most of them are far too verbose and unfocused.
- Prepare your presentation carefully. A messy speech with too much code or too many files makes it nearly impossible for the audience to follow. Think in advance about what you want to highlight
- You should thoroughly analyze the training/test split. You should also use the same splits for different approaches so that the results can be accurately compared. And if there is a huge mismatch between CV vs kaggle LB - consider using adversarial validation (**For Option 1**).
- Your final report MUST include both CV and LB scores for all experiments (**For Option 1**).

Grading Policy and Defense

Grading Policy is outlined in the table

Criterion	Points
Overall correctness and completeness of the submitted code for the lab assignment, overall quality (metrics) of the proposed solution	5
Practical question about the submitted code during defense	7
Theoretical part during defense (2 questions + 1 exercise)	5
Criterion	Points

The defense will be conducted during a practical session, where the Lecturer will evaluate your code, report, and presentation skills. It is important to emphasize that the assessment will not be limited to metrics, analysis, and the performance of your models, but will also take into account the overall quality of your code, including readability, structure, and adherence to good practices. During the defense, the Lecturer will evaluate each student's individual contribution, demonstrated skills, and depth of

knowledge. In addition, up to 2 theoretical questions may be asked; these will be based on the questions outlined in Appendix 1, but will not necessarily be restricted to them.

Exercise with practical tasks similar to Appendix 2 will be evaluated in the form of a written test on one of the practical sessions

AI Tools usage policy

Scope

This policy applies to all generative AI tools that produce or modify text, code, images, or other digital content. This includes, but is not limited to:

- Text generation tools: ChatGPT, Claude, Gemini, Mistral, LLaMA-based models, etc.
- Code generation tools: GitHub Copilot, OpenAI Codex, TabNine, etc.
- Language assistance tools: Grammarly, DeepL, Google Translate, etc.
- Local LLMs or self-hosted AI models.

Allowed Uses

You may use AI tools for the following purposes, provided they comply with the conditions below:

- Spell Check & Style Correction. Example: Using Grammarly to correct grammar and improve style in a lab report or paper review.
- Translation. Example: Using DeepL or Google Translate to translate sections of a document into another language.
- Idea Generation & Pitching. Example: Brainstorming possible approaches for solving a problem or generating research ideas.
- Self-Education. Example: Using ChatGPT to explain an unfamiliar algorithm or concept.
- Library & Documentation Exploration. Example: Asking AI to summarize PyTorch documentation for a specific class or function.
- Code Assistance (Partial). Writing or correcting specific functions or methods (not entire solutions). Examples: Write a function that reads data and converts it to another format. Implement a data collator with padding to the longest sequence. Create an image augmentation function for training.

Conditions for Use

Full Transparency:

- Any use of AI tools must be disclosed in your submission.
- Include all prompts, responses, and/or screenshots showing your interaction with the AI tool.
- If using online tools, include chat links (if available) or full transcripts in an appendix of your report.

Partial Contribution Only:

- AI tools should support your work — they must not produce the complete assignment, lab, or report.
- You must be able to explain and defend any AI-generated content during oral defense or Q&A

Examples

Acceptable Uses:

- "Correct grammar and style in this paragraph of my report."
- "Write a function to load images from a directory and resize them."

- *"Suggest 3 ideas for improving model generalization."*

Unacceptable Uses:

- *"Write the entire peer review for this paper."*
- *"Generate the complete solution code for this assignment task."*

Appendix 1 (Theoretical Questions)

- *Explain why we need the main CNN building blocks: convolution, pooling, normalization, and activation layers. What problem does each component solve in representation learning?*
- *Explain the fundamental differences between Speaker Verification, Speaker Identification, Speaker Tracking, and Speaker Diarization in terms of input constraints, expected output (binary vs. multi-class classification), and supervision (supervised vs. unsupervised learning).*
- *Why is Speaker Diarization inherently formulated as an unsupervised clustering problem rather than a standard supervised classification task?*
- *How does speaker diarization complement and enhance downstream speech processing tasks, such as Automatic Speech Recognition (ASR) and Speech Emotion Recognition (SER), in multi-speaker conversational environments?*
- *What are the core components of a standard sequential Speaker Diarization pipeline? Detail the individual roles of the segmentation module, the audio embedding extractor, and the clustering algorithm.*
- *Explain the specific evaluation challenges in Speaker Diarization. Why must an optimal speaker mapping algorithm (e.g., the Hungarian algorithm) be applied to align the inference labels with the ground truth before computing error rates?*
- *In the context of the ESC-50 environmental sound dataset, explain the significance of the `src_file` and `take` metadata fields. Why is it critical to group by the original source file when creating cross-validation folds?*
- *Explain how random or naive stratified splitting without grouping by the original acoustic source (e.g., using standard `StratifiedKFold` instead of `StratifiedGroupKFold`) leads to severe data leakage in audio classification tasks.*
- *Detail the signal processing transformations required to convert a raw 1D audio waveform into a 2D Mel-spectrogram. What is the mathematical and perceptual justification for applying an `amplitude_to_DB` (logarithmic) scaling step after applying the Mel filterbank?*
- *Compare and contrast waveform-level augmentations (such as Gaussian noise addition and time stretching via `audiomentations`) with spectrogram-level augmentations (such as `FrequencyMasking` and `TimeMasking`). What specific representation regularizations do `SpecAugment` techniques provide?*
- *Explain the rationale behind using 2D image-based neural network architectures (accessed via the `timm` library) for audio classification tasks. How does the network interpret the distinct time and frequency axes of a Mel-spectrogram compared to the spatial height and width dimensions of a standard RGB image?*
- *When evaluating a multi-class environmental sound classifier across 50 classes, why is it necessary to compute macro-averaged precision, recall, and F1-score alongside basic accuracy?*
- *How does the `pyannote` framework handle the end-to-end processing of audio for diarization, and what is the conceptual purpose of fine-tuning its pre-trained segmentation module on a specific target dataset?*
- *In PyTorch dataloaders for continuous speech tasks, audio clips often have variable temporal durations. Explain the role of a custom `collate_fn`. How does zero-padding the temporal*

dimension of a spectrogram to create uniform batch tensors affect the convolutions, and why might attention masks be required later in the network?

- *Contrast the application of Batch Normalization versus Instance Normalization on Mel-spectrograms. Since the frequency axis represents distinct physical acoustic bins (unlike spatial pixels in an image), how does normalizing across the batch versus normalizing across the time/frequency dimensions impact the network's ability to learn pitch invariance?*
- *Explain the low-level mechanics of Permutation Invariant Training (PIT) loss used in neural segmentation models. How does PIT dynamically compute the minimum Binary Cross-Entropy (BCE) loss across all possible speaker label permutations to effectively handle overlapping speech without relying on a fixed speaker order?*
- *Contrast the final activation functions and loss objectives required for the ESC-50 task versus a frame-level Voice Activity Detection (VAD) model. Why does the ESC-50 multi-class classifier utilize Softmax with Categorical Cross-Entropy, while a multi-speaker segmentation model must use Sigmoid activations with Binary Cross-Entropy?*
- *When fine-tuning a pre-trained timm backbone or a pyannote segmentation model, explain the concepts of gradient freezing and differential learning rates. At a computational graph level, what happens to the requires_grad attributes and the backward pass when we freeze the early feature-extraction layers while actively training the final classification head?*
- *In speaker verification and diarization networks, how does a statistical pooling layer operate at the end of the convolutional feature extractor? Explain the mathematical difference between standard Global Average Pooling (used in standard vision models) and computing both the mean and standard deviation across the temporal dimension to capture variable-length speaker characteristics.*

Appendix 2 (Practical Exercises)

Audio ML Lab №2 — Practical Exercises

Exercise 1: 2D Convolution Operation

Consider a single-channel 3×3 input image X and a 2×2 convolutional kernel W :

$$X = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 3 & 2 \\ 0 & 1 & 1 \end{bmatrix}, \quad W = \begin{bmatrix} 1 & -1 \\ 0 & 2 \end{bmatrix}$$

Assume the following layer parameters:

- Stride: $s = 1$
- Padding: $p = 0$
- Bias: $b = 1$

Tasks

1. Compute the spatial dimensions (height and width) of the resulting output feature map.
2. Perform the forward pass to compute the exact values of the output feature map after the convolution operation (including the bias term).

Exercise 2: Permutation Invariant Loss

Speaker diarization systems often need to predict multiple speaker activity tracks simultaneously. Since the ordering of speakers is arbitrary, the loss function must be invariant to permutations of the outputs. In practice, speaker diarization models commonly use a **Permutation Invariant Training (PIT)** objective together with a **Binary Cross-Entropy (BCE)** loss.

For simplicity, in this exercise we will study the same idea using a Mean Squared Error (MSE) loss on scalar predictions.

Consider a ground truth set of two scalar values $Y = \{y_1, y_2\}$ and a predicted set $\hat{Y} = \{\hat{y}_1, \hat{y}_2\}$.

A standard Mean Squared Error (MSE) loss assumes a fixed alignment, which penalizes correct predictions that are merely out of order. To address this issue, we use a permutation invariant loss based on bipartite matching.

Given:

$$Y = \{2.0, 5.0\}, \quad \hat{Y} = \{4.5, 1.5\}$$

Tasks

1. Define a mathematical formulation for a permutation invariant MSE loss $\mathcal{L}_{inv}(Y, \hat{Y})$ for a set of size $N = 2$.
2. Calculate the standard ordered MSE loss assuming the pairing $(\hat{y}_1 \leftrightarrow y_1)$ and $(\hat{y}_2 \leftrightarrow y_2)$.
3. Calculate the permutation invariant loss $\mathcal{L}_{inv}$ using your formulation from Task 1.

Exercise 3: Detection Error Rate (DER)

Speaker Diarization aims to determine *who spoke when* in an audio recording. A commonly used evaluation metric is the **Diarization Error Rate (DER)**.

For simplicity, consider a recording divided into 12 equal-length segments.

The ground truth speaker sequence is:

[A, A, B, B, S, A, A, B, S, S, B, A]

The diarization system predicts:

[A, B, B, B, A, A, S, B, S, A, A, A]

where:

- A — Speaker A
- B — Speaker B
- S — Silence

Assume that all segments have equal duration.

Use definition of DER from Lecture

Tasks

1. Determine the number of False Alarm (FA) errors.
2. Determine the number of Missed Detection (MD) errors.
3. Determine the number of Speaker Confusion (SC) errors.
4. Compute the Detection Error Rate (DER).

Exercise 4: Shape Computation in an Audio Classification Pipeline

Consider a standard audio classification pipeline:

1. **Spectral Conversion:** convert waveform audio into a Mel-spectrogram.
2. **CNN Backbone:** extract features and reduce spatial dimensions.
3. **Global Average Pooling:** average over spatial dimensions.
4. **Linear Classification Layer:** produce final class logits.

Assume the following input and model parameters:

- Batch size: $bs = 4$ samples
- Sample rate: $sr = 16000$ Hz
- Audio length: $T = 4$ seconds

- Hop size: $h = 250$ samples
- Number of Mel bins: $M = 64$
- CNN output channels: $C = 256$
- CNN spatial dimension reduction ratio: $r = 16$
- Number of classification classes: $K = 10$

Tasks

1. Compute the number of waveform samples N .
2. Compute the Mel-spectrogram shape $[M, F]$.
3. Compute the CNN backbone input shape.
4. Compute the CNN backbone output shape.
5. Compute the shape after Global Average Pooling.
6. Compute the final output shape after the linear classification layer.